

Chapter 6: Inheritance and Abstract Classes

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives

- Use inheritance to share code among a set of classes
- Customize the behavior of a class by creating a subclass
- Factor the redundant data and methods of a set of classes into an abstract class
- Override the behavior of an inherited method by redefining it
- Decide where to place data and methods in a class hierarchy

Using Inheritance to Customize an Existing Class

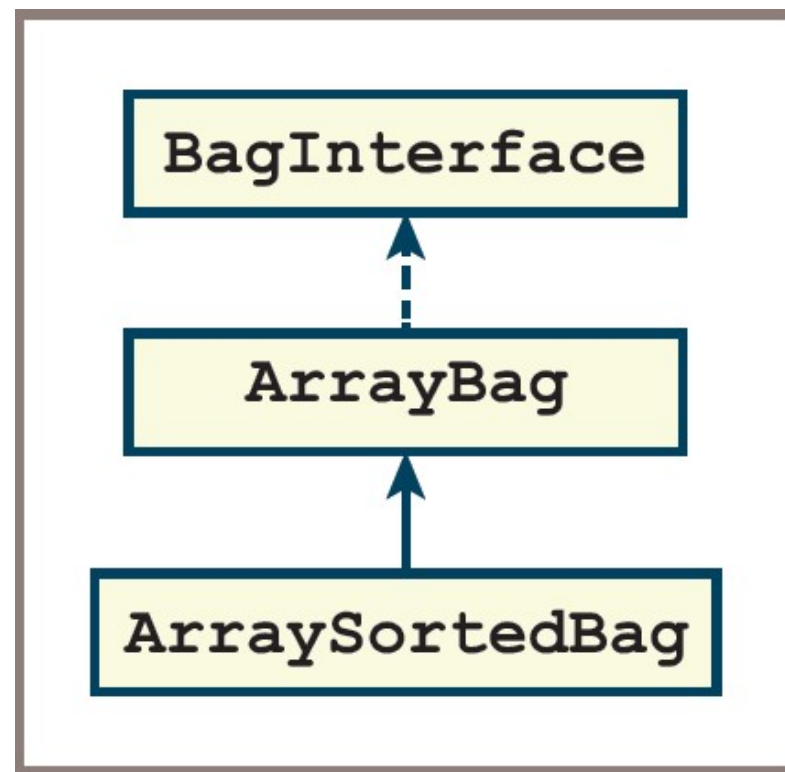
- Easiest way to take advantage of inheritance
 - Use it to customize an existing class
- Example: sorted bag class **ArraySortedBag** (Project 3 of Chapter 5)
 - The sorted bag behaves just like a regular bag, but with exceptions:
 - The sorted bag allows the client to visit its elements in sorted order via the `for` loop
 - The sorted bag's `in` operator runs in logarithmic time
 - The items added to the bag must be comparable with each other

Subclassing an Existing Class (1 of 3)

- This section covers inheritance by making the **ArraySortedBag** class a *subclass* of the **ArrayBag** class:
 - **ArrayBag** is called the parent or superclass of **ArraySortedBag**
- Note
 - Because the **ArrayBag** class implements **BagInterface**, the **ArraySortedBag** class also implements this interface via inheritance

Subclassing an Existing Class (2 of 3)

Figure 6-1: Depicting a subclass and inheritance in a class diagram



Subclassing an Existing Class (3 of 3)

- Creating a subclass of an existing class consists of working on a copy of the parent's class file and performing the following steps:
 - Begin by deleting all the methods that do not have to change:
 - You still need the `_init_` method in the new class
 - To guarantee that the inheritance happens, you must place the name of the parent class within the parentheses of the class header
 - Modify the code for the methods that must change
 - Add any new methods

Revising the `_init_` Method (1 of 2)

- There is one case where inheritance is not automatic
- The `_init_` method in **ArraySortedBag** must call the `_init_` method in the parent class **ArrayBag**
 - So that it can initialize the data contained there
- The syntax to call this method in the parent class is `ArrayBag._init_(self, sourceCollection)`
- The parent class name enables Python to select which version of the `_init_` method to run
- Note the additional argument, `self`, at the beginning of the argument list
- The optional source collection is also passed as an argument to the parent class's `_init_` method

Revising the `__init__` Method (2 of 2)

- Code for the changes to `ArraySortedBag` so far:

```
"""
```

```
File: arraysortedbag.py
```

```
Author: Ken Lambert
```

```
"""
```

```
from arraybag import ArrayBag
```

```
class ArraySortedBag(ArrayBag):
```

```
    """An array-based sorted bag implementation."""
```

```
    # Constructor
```

```
    def __init__(self, sourceCollection = None):
```

```
        """Sets the initial state of self, which includes the  
        contents of sourceCollection, if it's
```

```
present."""
```

```
        ArrayBag.__init__(self, sourceCollection)
```

Adding a New `_contains_` Method (1 of 2)

- There is no `_contains_` method in **ArrayBag**:
 - Python automatically generates a sequential search operation, using the **ArrayBag** iterator, when the `in` operator is used on a bag
- To override this behavior, include a `_contains_method` in **ArraySortedBag**:
 - When Python sees the `in` operator used on a sorted bag, it also sees that bag's `_contains_method` and calls it
- This method implements a binary search on the sorted bag's array of items:
 - This array, named `self.items` is accessible in any of its subclasses
 - You can reference this variable directly during the search

Adding a New `__contains__` Method (2 of 2)

- Code:

```
# Accessor methods
def __contains__(self, item):
    """Returns True if item is in self, or False
    otherwise."""
    left = 0
    right = len(self) - 1
    while left <= right:
        midPoint = (left + right) // 2
        if self.items[midPoint] == item:
            return True
        elif self.items[midPoint] > item:
            right = midPoint - 1
        else:
            left = midPoint + 1
    return False
```

Modifying the Existing add Method (1 of 4)

- The **add** method in **ArraySortedBag** must place a new item in the appropriate position in a sorted array
- In most cases you have to search for this position
- Two cases in which this search is unnecessary:
 - When the bag is empty
 - When the new item is greater than or equal to the last item
- In those cases you can add the new item by passing it along to the **add** method in the **ArrayBag** class
- If not, you must look in the array for the first item that's greater than or equal to the new item:
 - Then open a hole, insert the new item, and increment the size of the bag

Modifying the Existing add Method (2 of 4)

- Code:

```
# Mutator methods
def add(self, item):
    """Adds item to self."""
    # Empty or last item, call ArrayBag.add
    if self.isEmpty() or item >= self.items[len(self) - 1]:
        ArrayBag.add(self, item)
    else:
        # Resize the array if it is full here
        # Search for first item >= new item
        targetIndex = 0
        while item > self.items[targetIndex]:
            targetIndex += 1
        # Open a hole for new item
        for i in range(len(self), targetIndex, -1):
            self.items[i] = self.items[i - 1]
        # Insert item and update size
        self.items[targetIndex] = item
        self.size += 1
```

Modifying the Existing add Method (3 of 4)

- Note the different prefixes used in the method call to **ArrayBag.add** and in the variable reference to **self.items**:
 - The first is a class name, and the second is a reference to an instance of a class.
 - Because **ArraySortedBag** does not introduce a new version of the instance variable **items**, the reference to **self.items** here locates the variable directly in the **ArrayBag** class

Modifying the Existing add Method (4 of 4)

- The `__add__` method, which Python runs when it sees the `+` operator used with two bags, has practically the same code in the **ArrayBag** class and the **LinkedBag** class:
 - The only difference is the class name used to create a new instance for the result bag
 - You simply repeat this pattern for `__add__` in **ArraySortedBag**, as follows:

```
def __add__(self, other):  
    """Returns a new bag with the contents of self and other."""  
    result = ArraySortedBag(self)  
    for item in other:  
        result.add(item)  
  
    return result
```

Run-Time Performance of `ArraySortedBag` (1 of 2)

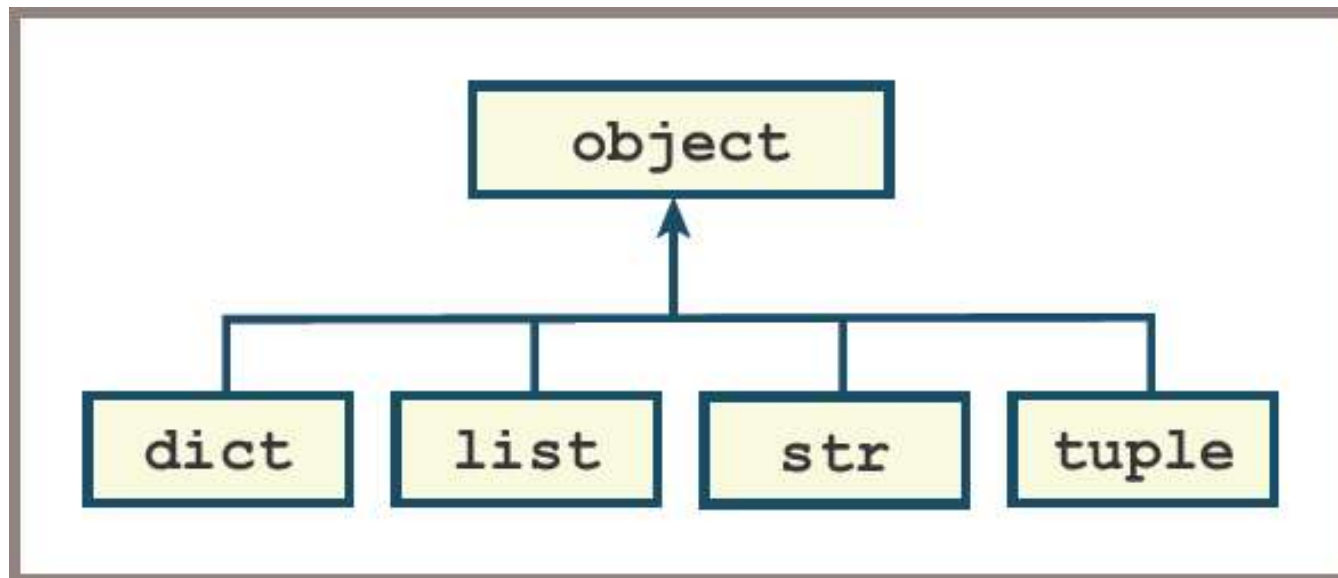
- The `in` operator has a worst-case running time of $O(\log n)$:
 - This is a big improvement on the linear running time of the `in` operator for regular bags
- The improved search time also has an impact on the methods that use the new `in` operator,
 - Even though these methods are still defined in `ArrayBag`
- Although the improved search still leaves `remove` with a linear running time on average,
 - Less time is spent on the test of the precondition

Run-Time Performance of `ArraySortedBag` (2 of 2)

- The performance improvement in the `__eq__` method can be more dramatic,
 - Even though it does not use the `in` operator
- When this method is run on two regular bags of the same length, its average running time is $O(n^2)$
- If you are willing to redefine the `__eq__` method in **`ArraySortedBag`**,
 - You can reduce its average running time to $O(n)$
- This implementation takes advantage of the fact that the pairs of items being compared in the two sorted bags appear in the same order as you iterate through them

A Note on Class Hierarchies in Python (1 of 2)

Figure 6-2: A portion of Python's class hierarchy



A Note on Class Hierarchies in Python (2 of 2)

- When you define a new class and omit the parent class from the syntax,
 - Python automatically installs this class under the **object** class
- When you design a new set of collection classes,
 - It's generally not a good idea to subclass under a built-in Python collection class
- Instead, develop your own classes to conform to interfaces
 - And place the topmost class in your hierarchy under the **object** class

Using Abstract Classes to Eliminate Redundant Code

- Keep potentially redundant methods in a parent class and share them with another class via inheritance
- This section covers how to eliminate redundant methods and data in a set of existing classes by factoring the code for them into a common superclass:
 - Called an *abstract class*, it captures the common features and behavior of a set of related classes
- Its subclasses are called *concrete classes* to indicate that they are the classes actually used to create objects in client applications

Designing an AbstractBag Class (1 of 4)

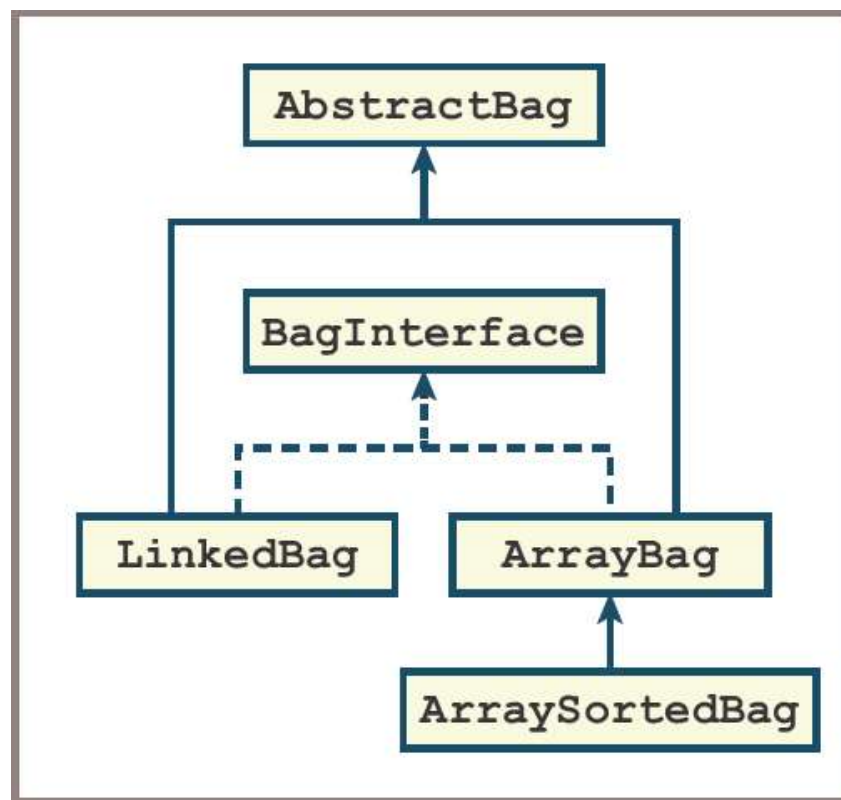
- In the case of your bag classes, the most redundant methods include:
 - `isEmpty`, `_str_`, `_add_`, `count`, and `_eq_`
- Redundant instance variables are trickier to spot:
 - Your bag classes use two instance variables: `self.items` and `self.size`
- To discovery redundancy:
 - Look at the types of data to which the variables refer
 - Only `self.size` is redundant as it refers to an integer value in every bag class (safe to move to an abstract class)
- Because the `_len_` method accesses `self.size` but not `self.items`:
 - It counts as a redundant method

Designing an **AbstractBag** Class (2 of 4)

- You can remove redundant methods from the bag classes and place them in a new class called **AbstractBag**
- The bag classes then access these methods via inheritance by becoming subclasses of **AbstractBag**
- Note that the **AbstractBag** class does not implement the bag interface:
 - You also now have a more obvious class hierarchy, with two levels of inheritance
- The **ArraySortedBag** class now inherits some methods and data directly from its parent class **ArrayBag**
 - And other methods and data indirectly from its ancestor class **AbstractBag**

Designing an AbstractBag Class (3 of 4)

Figure 6-3: Adding an abstract bag class to the collection framework



Designing an AbstractBag Class (4 of 4)

- To create the **AbstractBag** class, you start by copying the contents of one of its subclasses to a new file and save that file as **abstractbag.py**
- Then perform the following steps:
 - Delete any irrelevant imports and rename the class **AbstractBag**
 - Delete all the methods that directly access the instance variable **self.items**, except for the **__init__** method:
 - The next subsection shows you how to modify the **__init__** method

Redoing the `__init__` Method in `AbstractBag` (1 of 2)

- The `__init__` method in `AbstractBag` is responsible for performing two steps:
 - Introducing the variable `self.size` and initializing it to 0
 - Adding the items from the source collection to `self`, if the source collection is present
- Therefore, you delete the line of code that initializes the variable `self.items`:
 - This code is still the responsibility of the `__init__` method in the subclasses

Redoing the `__init__` Method in `AbstractBag` (2 of 2)

- Code:

```
"""
```

```
File: abstractbag.py
```

```
Author: Ken Lambert
```

```
"""
```

```
class AbstractBag(object):
```

```
    """An abstract bag implementation."""
```

```
    # Constructor
```

```
    def __init__(self, sourceCollection=None):
```

```
        """Sets the initial state of self, which includes the  
        contents of sourceCollection, if it's present."""
```

```
        self.size = 0
```

```
        if sourceCollection:
```

```
            for item in sourceCollection:
```

```
                self.add(item)
```

Modifying the Subclasses of **AbstractBag** (1 of 3)

- Each subclass of **AbstractBag** must now
 - Import this class
 - Place its name within the parentheses in the class header
 - Omit the redundant methods mentioned earlier
 - Include a modified **`_init_`** method
- Examine the changes to the **`_init_`** method of **ArrayBag**:
 - This method is still responsible for setting `self.items` to a new array
 - That's the only line of code kept from before
- Run the **`_init_`** method
 - Initializes the bag's size and adds the items from the source collection, if necessary

Modifying the Subclasses of AbstractBag (2 of 3)

- Code for the changes to `__init__` in `ArrayBag`:

```
"""
File: arraybag.py
Author: Ken Lambert
"""

from arrays import Array
from abstractbag import AbstractBag

class ArrayBag(AbstractBag):
    """An array-based bag implementation."""

    # Class variable
    DEFAULT_CAPACITY = 10

    # Constructor
    def __init__(self, sourceCollection = None):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        self.items = Array(ArrayBag.DEFAULT_CAPACITY)
        AbstractBag.__init__(self, sourceCollection)
```

Modifying the Subclasses of `AbstractBag` (3 of 3)

- Note the order in which the two statements are written in `__init__`
- It is critical to initialize `self.items` to the new array before running the constructor in the superclass
 - So that there will be storage for any items that are added to the new bag

Generalizing the `__add__` Method in `AbstractBag` (1 of 2)

- If you test the bag classes with your tester function at this point,
 - The `+` operator (which uses the `__add__` method in `AbstractBag`) raises an exception
 - The exception states that `AbstractBag` does not know about `ArrayBag` (or `LinkedBag` or `ArraySortedBag`)
- The cause of this error is that the `__add__` method has attempted to create an instance of `ArrayBag` to hold its results
 - As shown in the next code segment:

```
def __add__(self, other):  
    """Returns a new bag containing the contents  
    of self and other."""  
    result = ArrayBag(self)  
    for item in other:  
        result.add(item)  
    return result
```

Generalizing the `__add__` Method in `AbstractBag` (2 of 2)

- To solve this problem,
 - Use Python's `type` function to access the type of `self`
 - Then use the resulting type to create a clone of `self` in the usual manner
- Code for an `__add__` method that works with all bag types:

```
def __add__(self, other):  
    """Returns a new bag containing the contents  
    of self and other."""  
    result = type(self)(self)  
    for item in other:  
        result.add(item)  
    return result
```

An Abstract Class For All Collections

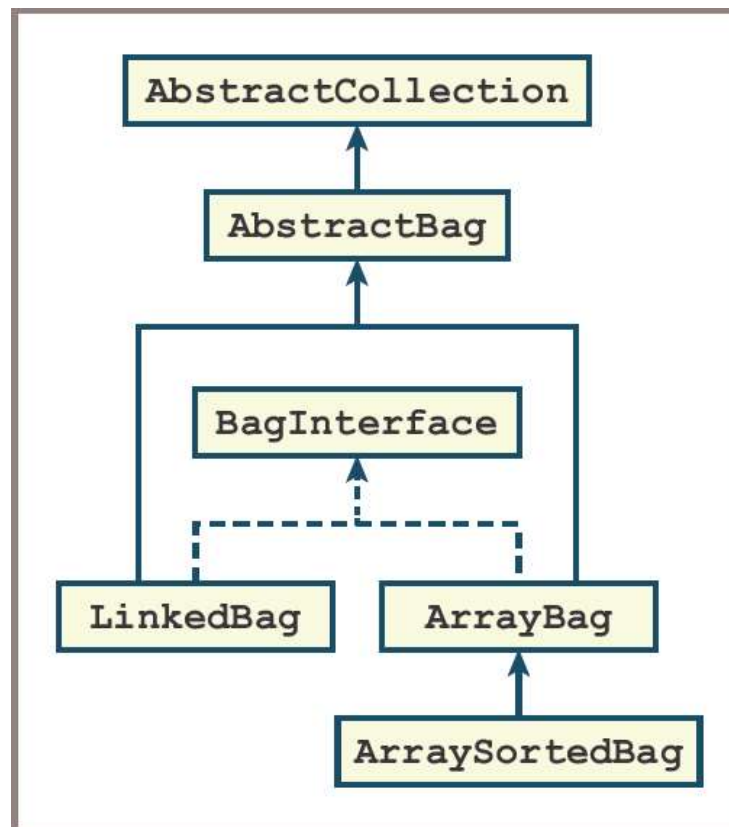
- Review the code for the **AbstractBag** and note the following:
 - All of its methods, including `__init__`, run other methods or functions or simply access the variable `self.size`
 - They make no mention of bag classes
 - With the exception of the `__str__` method, which creates a string with curly braces, and the `__eq__` method, which does not compare pairs of items at given positions
 - The **AbstractBag**'s methods are also methods that could be run on any other types of collections, such as lists, stacks, and queues
 - Finally, the one instance variable, `self.size`, could be used in the implementation of any collection as well

Integrating **AbstractCollection** into the Collection Hierarchy (1 of 3)

- The **AbstractCollection** class is responsible for introducing and initializing **self.size**:
 - This variable is used by all the collection classes below it in the hierarchy
- The **__init__** method of **AbstractCollection** can also add the items from the source collection to **self** (if necessary)
- This class also includes the most general methods available to all collections:
 - **isEmpty**, **__len__**, **count**, and **__add__**
- **AbstractCollection** also includes default implementations of the **__str__** and **__eq__** methods:
 - Their current form in **AbstractBag** is appropriate for unordered collections, but most collection classes are likely to be linear rather than unordered

Integrating AbstractCollection into the Collection Hierarchy (2 of 3)

Figure 6-4: Adding an abstract collection class to the collection framework



Integrating **AbstractCollection** into the Collection Hierarchy (3 of 3)

- To create the **AbstractCollection** class, copy code as usual from another module, in this case, **AbstractBag**
- You can now perform the following steps:
 - Rename the class to **AbstractCollection**
 - Modify the `__init__` method by removing the call to `__init__` in a parent class
 - Modify the `__str__` and `__eq__` methods to provide reasonable default behavior
 - Then remove the `isEmpty`, `__len__`, and `__add__` methods from **AbstractBag**

Using Two Iterators in the `_eq_` Method (1 of 3)

- The implementation of the `_eq_` method in the **`AbstractCollection`** class compares pairs of items in the two collections
- The following two code segments perform the same task, but the first one uses Python's `for` loop and the second one directly manipulates the collection's iterator object:

```
# Print all the items in theCollection using a for loop
for item in theCollection:
    print(item)
```

```
# Print all the items in theCollection using an explicit iterator
iteratorObject = iter(theCollection)
try:
    while True:
        print(next(iteratorObject))
except StopIteration: pass
```

Using Two Iterators in the `_eq_` Method (2 of 3)

- When you employ an explicit iterator in the `__eq__` method of **AbstractCollection**:
 - There is no need to trap a **StopIteration** exception
- Because the two collections have the same length, the **for** loop on the first collection will stop when the iterator on the second collection reaches the end of its sequence of items

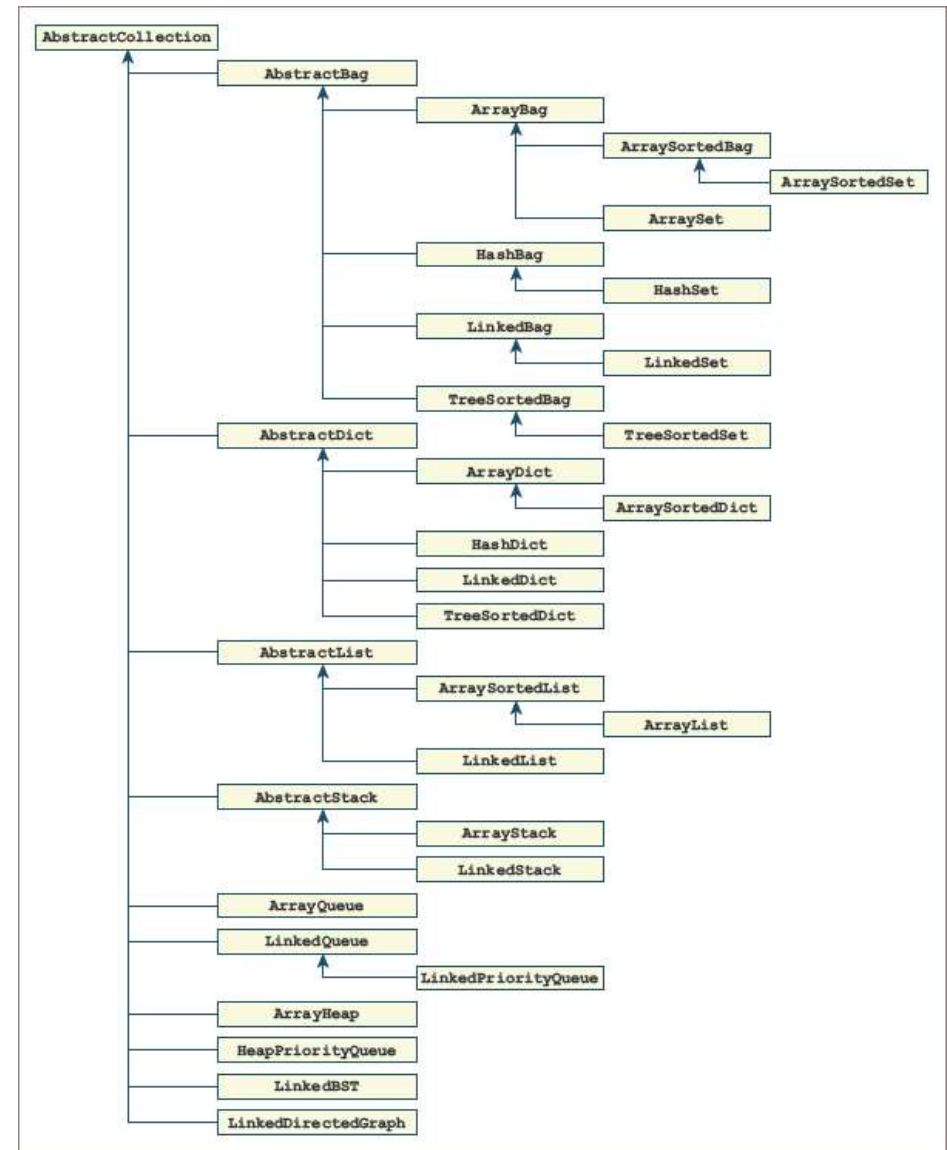
Using Two Iterators in the `_eq_` Method (3 of 3)

- Code for the new `_eq_` method

```
def __eq__(self, other):  
    """Returns True if self equals other, or False  
    otherwise."""  
    if self is other: return True  
    if type(self) != type(other) or \  
        len(self) != len(other):  
        return False  
    otherIter = iter(other)  
    for item in self:  
        if item != next(otherIter):  
            return False  
    return True
```

A Professional-Quality Framework of Collections

Figure 6-5: A professional-quality collection framework



Chapter Summary (1 of 2)

- Two classes can be related as subclass and superclass:
 - A subclass generally is a more specialized version of its superclass
 - The superclass is also called the parent of its subclasses
- A subclass inherits all the methods and variables from its parent class, as well as any of its ancestor classes
- A subclass specializes the behavior of its superclass by modifying its methods or adding new methods
- A class can call a method in its superclass by using the superclass name as a prefix to the method
- An abstract class serves as a repository of data and methods that are common to a set of other classes

Chapter Summary (2 of 2)

- Abstract classes are not instantiated
- Python includes all classes in a hierarchy, with the **object** class as the topmost parent class
- The methods with more general behavior should be located further up in a class hierarchy